EXPERIMENT 6

PRODUCER–CONSUMER PROBLEM USING SEMAPHORES

AIM: To implement the Producer–Consumer problem using semaphores for process synchronization in the Linux operating system.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

PROCEDURE:

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Include the required header files for threads and semaphores.
4. Initialize the empty and full semaphores.
5. Create producer and consumer threads.
6. The producer inserts items into the shared buffer.
7. The consumer removes items from the shared buffer.
8. Use semaphores to prevent buffer overflow and underflow, and a mutex to protect buffer access.
9. Display the produced and consumed items.
10. Compile, execute and verify the output.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>

int mutex=1,full=0,empty=3,x=0;

void producer();
void consumer();
int wait(int);
int signal(int);

int main()
{
    int n;

    printf("\n1.Producer\n2.Consumer\n3.Exit");

    while(1)
    {
        printf("\nEnter your choice:");
        scanf("%d",&n);

        switch(n)
```

```
        {
            case 1:
                if((mutex==1)&&(empty!=0))
                    producer();
                else
                    printf("Buffer is full!!");
                break;

            case 2:
                if((mutex==1)&&(full!=0))
                    consumer();
                else
                    printf("Buffer is empty!!");
                break;

            case 3:
                exit(0);
                break;
        }
    }

    return 0;
}

int wait(int s)
{
    return (--s);
}

int signal(int s)
{
    return (++s);
}

void producer()
{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;

    printf("\nProducer produces the item %d",x);

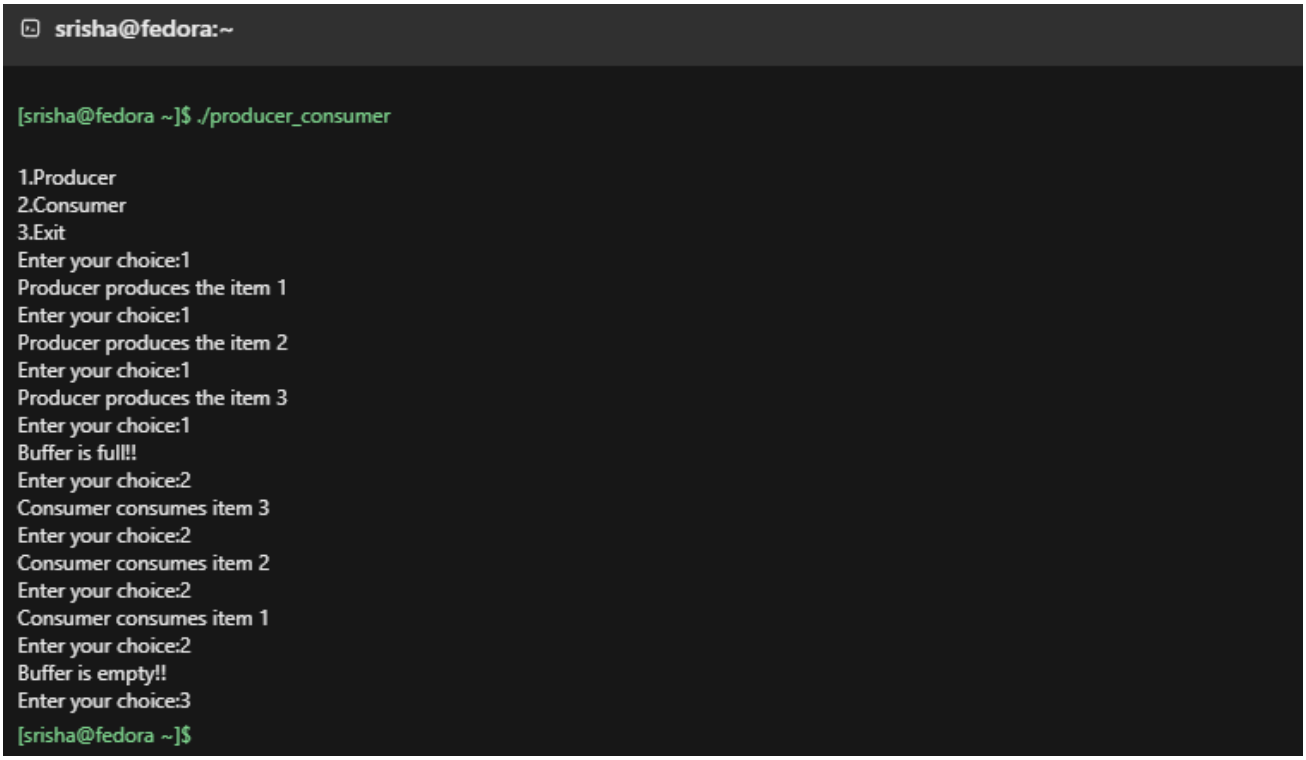
    mutex=signal(mutex);
}

void consumer()
{
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
```

```
    printf("\nConsumer consumes item %d",x);
    x--;

    mutex=signal(mutex);
}
```

OUTPUT:



RESULT:

Thus, the Producer–Consumer problem was implemented and executed successfully using simulated semaphore operations in C.